정연서 포트폴리오

Forward Deployed Engineer · Engineering × Strategy × Consulting

yeonseosla@gmail.com github.com/yeonseo-Jung linkedin.com/in/yeonseo-jung yeonseo-jung.github.io

목차

0. 전략·컨설팅 — 고객 진단, 범위 수립, 협상, 표준화
1. 13년치 레거시 ERP를 손실 없이 운영 이관 — 시험·인증기관
2. 기능 구현에서 업무 구조 전환으로 — 4개 법인 제조·유통 그룹 ERP
3. AI 에이전트가 ERP를 안전하게 쓰게 하는 통제 아키텍처
4. 흩어진 6개 원천을 자연어로 묻는 AI 데이터 워크스페이스 — 소비재 브랜드
5. 유지보수 요청 LLM 트리아지 — 데이터로 정한 AI 도입 속도
6. 사내 시스템을 에이전트가 쓰게 하는 헤드리스 접근 계층
7. 1,600+ 계좌 24/7 분산 주문 실행 인프라와 Rust 전환
8. 레거시 데이터 파이프라인을 Airflow 플랫폼으로 전환
9. 서비스 매출 40%에 기여한 구독형 운용 상품 — 기획부터 엔진까지

0–6은 위시켓 AIDP에서 고객 현장에 들어가 진단부터 운영 전환까지 맡은 사례, 7–9는 STOA Investment에서 상품과 인프라를 처음부터 만든 사례입니다. 구현은 Claude Code · Codex 에이전트 팀을 설계·지휘하는 방식으로 진행하고, 사람의 시간은 진단과 의사결정에 씁니다.

0. 전략·컨설팅 — 고객 진단, 범위 수립, 협상, 표준화

위시켓 AIDP · 2026.06–현재

- **고객 인터뷰와 목표 수립** — 소비재 브랜드 프로젝트에서 팀별 심층 인터뷰 질의 은행과 진행 키트를 설계하고, 인터뷰 결과를 팀별 핵심 목표 · 마일스톤 · 범위 밖 항목으로 좁혀 MVP 범위를 정했습니다.
- **클라우드 아키텍처·견적 협상** — 고객 AWS 환경을 MSP와 함께 설계하면서 아키텍처를 v13 → v21로 반복 개정하고, 견적서의 미팅 결론과 다른 가정(로그 크기, 동기화 방식, 비밀값 운영, 마이그레이션 위치)을 짚어 재산출을 요청했습니다.
- **문서 표준** — 제안서 · 견적서 · 킥오프 · 중간보고 · 완료보고 · 주간보고 · 전략서 7종을 같은 JSON 입력에서 PPTX · PDF로 생성하는 사업부 문서 표준을 만들어 사내 플러그인으로 배포했습니다.
- **에이전트 운영 체계** — 머신 3대에 걸친 개발·에이전트 운영 환경을 하나의 정본으로 관리하고(가드 훅, 승인 게이트, 교차 모델 리뷰), FDE가 에이전트에 일을 나누는 단위·모델 라우팅·자율 경계를 프로세스로 정리했습니다.
- **팀 기준** — FDE 채용 1차 면접관(2026-10~)을 맡고, 사업부 FDE의 역할·시간 배분에 대한 논의 자료를 작성했습니다.

1. 13년치 레거시 ERP를 손실 없이 운영 이관 — 시험·인증기관

위시켓 AIDP · 2026.06–2026.08 · 냉동공조 분야 시험·인증기관 · 이관 전략·런북·ETL·검증·고객 보고 리드

문제

2013년부터 쌓인 시험 이력이 레거시 SQL Server에 있었고, 신규 ERP(Next.js · Supabase/PostgreSQL)로 옮겨 운영을 전환해야 했습니다. 운영 환경은 하나뿐이어서 이관 실패가 곧 고객 업무 중단이었습니다.

설계 결정

- **게이트 분리** — Docker MSSQL 2022로 만든 로컬 랩에서 G0–G5 게이트를 모두 통과한 뒤에만 운영 DB(G6–G7)를 건드리도록 순서를 고정했습니다.
- **사이드맵** — 앱 스키마에 레거시 컬럼을 추가하지 않고 `migration_id_map` 테이블로 레거시 키 ↔ 신규 키를 추적했습니다. 앱 코드에 이관 흔적이 남지 않고, 행 단위 대조와 재실행이 가능해집니다.
- **4단계 검증** — 건수 대조 → 금액 합계 → 사이드맵 기준 행 단위 대조 → FK·enum 무결성. 검증기가 실제로 오류를 잡는지 확인하려고 변조 1건을 넣어 실패하는지 보는 역검증을 함께 돌렸습니다.

결과

항목	이관
시험의뢰	9,012건
시험일정	6,677건
성적서 발급 이력	3,722건
고객 회원 / 고객사	1,857명 / 1,107곳
첨부파일 (개별 무결성 검증)	1,564개

- 검증 31개 항목 전부 통과. 수수료 합계가 레거시와 1원 단위까지 일치. 레거시 64개 테이블 ↔ 신규 84개 테이블 매핑.
- 2026-07-03 운영 DB 본이관, 이후 병행 운영으로 전환.

이관 후 발견한 결함과 대응

이관 뒤 수수료 상태 4,359건이 뒤집혀 있는 것을 찾았습니다(입금완료 3,896건이 '수수료없음'으로, 수수료없음 463건이 '부분입금'으로). 변환 스크립트가 레거시 코드값의 의미를 화면 확인 없이 순서로 추측한 것이 원인이었습니다. 4단계 검증은 원본과 결과의 *값*을 비교하므로, 의미 매핑이 틀려도 통과합니다.

- 레거시 관리자 화면의 실제 select option 값으로 코드 의미를 확정하고 변환 상수를 고쳤습니다.
- 운영 정정은 대상 행 JSONL 백업 → 단일 트랜잭션 카나리 → 멱등 재실행 순서로 진행해 롤백 가능하게 했습니다.

SQL Server | PostgreSQL | Supabase | Python ETL | Next.js | Docker | Vitest | Playwright

2. 기능 구현에서 업무 구조 전환으로 — 4개 법인 제조·유통 그룹 ERP

위시켓 AIDP · 2026.07–2026.10 · 4개 법인 제조·유통 그룹 · ERP 오너(목표 재정의·Odoo 연동 설계·이관·릴리즈)

진단과 목표 재정의

Phase 1 고객 요구사항을 정리하고 기능·테스트·QA까지 마쳤지만, 고객의 업무 방식은 바뀌지 않았고 프로젝트의 필요성을 고객에게 증명할 근거도 약했습니다.

현장 재방문 반복 대응(요구 접수·분류·QA 요청)은 에이전트에 넘기고, 직접 고객사를 다시 찾아가 실제 업무를 확인했습니다. 담당자가 발주 포털과 엑셀을 오가며 채널별·건별로 처리하는 수작업이 병목이었습니다.

Phase 2 목표를 "요청 기능 구현"에서 "AI가 발주 처리를 수행하고 담당자는 결과만 승인하는 구조"로 다시 세우고, 이 전환이 고객 P&L 개선의 출발점이라는 논리로 고객에게 제안해 Phase 2를 시작했습니다.

기반 작업: 데이터 이관

법인별로 ECOUNT(SaaS ERP), 자체 WMS(MSSQL), 레거시 PHP ERP에 흩어진 거래·재고를 Odoo 19 기반 신규 ERP로 옮겨야 했습니다. 원장이 서로 다른 시스템 세 개라 "같은 데이터"의 기준부터 맞춰야 했습니다.

ECOUNT → Odoo 거래 이관

- 운영 반영 순서: 사전 점검(참조 ID 실재 확인) → 스냅샷 백업 → **운영 환경에서 롤백을 먼저 실연** → 거래처 동기화 → 전량 적재 → 다단계 검증 → 라인 단위 대사.
- 결과: 구매 2,027 + 판매 3,192 = **5,219문서 · 23,130라인**, 품목·수량·공급가액 전량 일치(불일치 0, 중복 0).
- VAT 반올림 차이 151라인은 원인과 영향 범위를 기록하고 수용 판정을 남겼습니다. 차이를 숨기지 않고 분류하는 것을 대사 원칙으로 두었습니다.
- 생산지시는 DB 직접 입력 대신 UI 경로로 이관했습니다. 생산지시 1건에 Odoo RPC 10개 이상이 엮여 있어, 직접 입력하면 Odoo 내부 상태와 어긋날 위험이 컸습니다.

기초재고 컷오버

- ECOUNT 209 + WMS 244 + 기적재 3 = 456개 재고 좌표(품목 · 위치 · 로트)를 두 원천에서 합쳐 적재했습니다.
- 재개 가능한 실행 원장, go/no-go 게이트, 롤백 지점, 산출물 해시를 갖춘 컷오버 스크립트를 작성했습니다.
- 결과: 기대값 대비 **456/456 일치**(초과·부족·누락·예상 외 0).

운영 절차

- 여러 담당자가 같은 통합 브랜치를 출하하면서 Slack 공지로 순서를 맞추는 방식이 3회 실패했습니다. 공지 대신 릴리즈 락(`release-lock acquire --head <검증한 SHA>`)을 잡아야 머지·배포할 수 있게 바꿨습니다.
- 레거시 PHP ERP에 하드코딩된 DB 자격증명을 환경변수로 옮기고 최소 권한 계정을 분리했습니다.

Odoo 19 JSON-RPC | Next.js | Prisma | Supabase | SQL Server | TypeScript | Vercel

3. AI 에이전트가 ERP를 안전하게 쓰게 하는 통제 아키텍처

위시켓 AIDP · 2026.08–2026.10 · 2번과 같은 ERP · 설계·구현 단독

문제

AI 에이전트가 ERP 데이터를 바꾸게 하려면, 모델 출력이 곧바로 DB 쓰기가 되는 구조를 막아야 했습니다. 잘못된 쓰기는 재고·회계에 바로 반영되고, 누가 무엇을 승인했는지가 남아야 합니다.

설계

- 모든 에이전트 쓰기를 **plan → authorization → commit** 3단계로 강제했습니다. 에이전트는 변경 계획만 만들고, 사람이 승인한 계획만 서버가 실행합니다.
- 승인은 Slack 카드로 받고, 카드 응답은 HMAC 서명으로 위변조를 막았습니다. 계획·승인·실행 기록은 원장 테이블에 남깁니다.
- 원장 스키마 마이그레이션 32개와, 제어면 코드(`l2/**` 79커밋, API 34커밋)를 단독으로 작성했습니다.

Next.js API Routes · PostgreSQL · HMAC · Slack API · Human-in-the-loop

4. 흩어진 6개 원천을 자연어로 묻는 AI 데이터 워크스페이스 — 소비재 브랜드

위시켓 AIDP · 2026.08–현재 · 다채널 유통 소비재 브랜드 · 현업 심층 인터뷰·목표 수립부터 아키텍처·구현·클라우드 협상까지 단독 리드

문제

판매·재고·정산 데이터가 ERP, 물류, 판매 채널 포털 등 6곳에 흩어져 있고, 담당자가 엑셀로 손으로 합치고 있었습니다. 목표는 원천을 한곳에 모으고, 현업이 자연어로 질문해도 틀린 숫자를 내놓지 않는 조회 경로를 만드는 것입니다.

설계 결정

- **모델은 SQL을 실행하지 않음** — LLM은 질문을 조회 계획(데이터셋 · 동작 · 조건)으로만 바꾸고, 서버가 카탈로그 기준으로 계획을 검증한 뒤 실행합니다. 답할 수 없는 질문과 데이터셋을 넘나드는 질문은 모델 판단과 별개로 서버가 차단합니다.
- **데이터 경계** — 원천 시스템은 읽기 전용으로만 연결하고, 계정·개인정보는 모델 입력에서 뺍니다. 고객 변경 요청의 판정은 규칙 엔진이 하고 LLM은 판정하지 않습니다. 모델 CLI는 도구·MCP·훅을 끈 격리 상태로 호출합니다.
- **저장소** — 운영 데이터는 PostgreSQL, 분석 질의는 DuckDB로 나누고, 두 엔진의 결과가 같은지 대조하는 테스트를 두었습니다.
- **모델 경로 교체** — 고객 AWS 계정에서 Bedrock Claude를 쓸 수 없다는 MSP 회신을 받고, 모델 경로를 Bedrock의 GPT 계열과 OpenAI Agents SDK로 바꿨습니다.

평가

자연어 조회 90문항(답 가능 · 모호 · 답 불가 · 교차 질문)으로 백엔드를 비교했습니다.

지표	Claude CLI	Codex CLI	규칙 기반
전체 정답률 (90문)	81.1%	87.8%	38.9%
action 어휘 제외 기준	93.3%	92.2%	—
답 불가·교차 질문 차단 (서버 강제)	100%	100%	100%

- 두 모델의 차이는 불일치 8쌍 기준 정확이항 양측 p = 0.070으로, 이 표본으로는 우열을 가를 수 없다고 판정했습니다. action 어휘를 빼면 순위가 뒤집힙니다.
- 오답 28건을 원인별로 나누자 가장 큰 유형은 값 하나를 묻는 질문에 집계 대신 행 나열을 고른 경우(15건)였습니다. 모델 교체보다 조회 계획 어휘와 서버 검증을 고치는 쪽이 효과가 크다는 근거가 됐습니다.
- 기본 모델은 정답률이 아니라 실행 경계의 잔여 위험을 우선하는 기준으로 정했고, 그 판단 근거를 정답률과 분리해 기록했습니다.

FastAPI · Next.js · PostgreSQL · DuckDB · AWS ECS · OpenAI Agents SDK · GitHub Actions · LLM Eval

5. 유지보수 요청 LLM 트리아지 — 데이터로 정한 AI 도입 속도

위시켓 AIDP · 2026.08–현재 · 2번 고객의 유지보수 티켓 대상 · 설계·구현 단독

문제

고객 유지보수 요청이 들어오면 사람이 매번 요구가 명확한지, 중복인지, 위험한지 판단한 뒤 착수했습니다. 이 판정과 답변 초안을 자동화해 착수 시간을 줄이되, 잘못된 자동 착수는 막아야 했습니다.

설계

- 30분 주기로 요청을 가져와 모호성 · 중복 · 난이도 · 리스크를 판정하고 답변 초안과 작업 워커 제안을 만듭니다. 판정마다 토큰·비용을 원장(SQLite)에 남기고 예산을 관리합니다.
- 판정 엔진은 실측 비용으로 골랐습니다(`claude -p` 호출당 약 $0.19 → 구독 기반 `codex exec`).
- 가드레일: dry-run 기본, 정지 마커, 근거 없는 판정 차단(근거 게이트), 발송 채널 허용 목록.

평가와 판단

- 과거 요청 204건 백테스트: 게이트 통과 4.4%, 사람 판단 대비 **과잉 차단 95.1%**, 과소 차단 0.
- 원인 분석: 204건 중 70%가 법인 판별 단계에서 범위 밖으로 빠졌고, 통과율을 바꾸는 축은 리스크 하나뿐이었습니다(단독 제거 시 4.3% → 11.2%). 거절 사례가 11%뿐이라 축별 검정력도 부족했습니다.
- 판단: 자동 착수를 밀어붙이는 대신 2일 시험 뒤 사람 판단과 나란히 돌리는 셰도우 단계로 도입 전략을 바꿨습니다. 13일 대조에서 방향 일치 70.3%를 확보하고, 규칙 개선 루프를 돌리고 있습니다.
- 규칙 변경은 같은 티켓을 재판정하는 A/B로 확인했습니다. 53쌍 중 리스크 상향 6건이 규칙 의도와 반대여서 개별 검증 대상으로 남겼습니다.

Python | SQLite | launchd | Codex CLI | Backtest | A/B

6. 사내 시스템을 에이전트가 쓰게 하는 헤드리스 접근 계층

위시켓 AIDP · 2026.07–2026.11 · 사내 딜리버리 어드민(Next.js · PostgreSQL) · 인증·에이전트 읽기 평면 설계

- **API 토큰** — 로그인 세션 없이 CLI·에이전트가 호출할 수 있게 토큰 인증을 추가했습니다. 토큰은 sha256 해시로만 저장하고 스코프·만료를 필수로 했습니다.
- **읽기 등급** — 데이터 영역마다 스코프를 새로 만들면 새 스코프가 기존 토큰에 소급되지 않아 토큰을 전량 재발급해야 했고, 이 일이 3회 반복됐습니다. 스코프 대신 읽기 등급을 두어 재발급 없이 범위를 넓힐 수 있게 바꿨습니다.
- **에이전트 읽기 평면** — 모델 176개를 민감도로 분류하고, 허용 컬럼만 담은 view 155개를 생성했습니다. PostgreSQL에서는 테이블 단위 SELECT를 준 뒤 특정 컬럼만 회수할 수 없어서, 권한 회수 대신 view로 노출 범위를 고정했습니다.
- 이 계층 위에 사내 Claude Code·Codex 플러그인용 CLI(`aidp-work`, 테스트 145개)를 만들었고, 계약 변경을 CI 게이트로 검사합니다.

TypeScript | PostgreSQL views | Token auth | CLI

7. 1,600+ 계좌 24/7 분산 주문 실행 인프라와 Rust 전환

STOA Investment · 2023–2026.06 · 실행 계층 설계·구현·운영 주도

문제

계좌가 수백에서 1,600개 이상으로 늘면서, 시그널 생성 · 계좌별 주문 판단 · 거래소 연동 · 체결 기록이 하나의 실행 흐름에 묶여 있어 확장성과 장애 격리가 약했습니다. 주문 메시지는 한 건이 곧 실제 자산 변경이라, 중복 실행과 유실이 모두 허용되지 않습니다.

설계 결정

- **경계 분리** — 시그널 생성은 상태 없이 두고, 주문 side effect와 계좌 상태는 실행 계층에서만 다루도록 나눴습니다. RabbitMQ fanout으로 시그널 1건을 계좌 N개에 배포합니다.
- **at-least-once + 멱등성** — publisher confirm, consumer manual ACK, DLQ로 전달을 보장하고, 중복 전달은 시간 슬롯 · 계좌 · 전략 · 시그널 ID를 묶은 멱등성 키로 흡수했습니다.
- **계좌 단위 bulkhead** — 주문 가능 수량, 포지션 동기화, 쿨다운을 계좌별로 격리해 한 계좌·거래소의 실패가 다른 계좌로 번지지 않게 했습니다.
- **관측** — MySQL을 주문·체결의 기준 기록으로, Redis는 가격 캐시로만 제한하고, Prometheus·Grafana로 처리량 · 지연 · DLQ · 계좌별 실패율을 추적했습니다.

Python → Rust 점진 전환

Celery와 asyncio가 섞인 Python 실행 계층은 24/7 코인 주문 영역에서 메모리와 이미지 크기, 런타임 일관성 문제가 커졌습니다. 코인 실행 계층만 전환 대상으로 정하고 Strangler Fig 방식으로 옮겼습니다. 두 구현이 RabbitMQ 경계에서 공존하고, 소비 주체는 feature flag로 바꾸고 되돌릴 수 있게 했습니다. 거래소 어댑터는 trait으로 계약을 고정하고, SQL은 sqlx 컴파일 타임 검증을 썼습니다.

항목	Python	Rust
런타임	Celery worker + asyncio	tokio, 계좌 단위 task
메모리	약 4.3GB	약 193MB (−95%)
Docker 이미지	약 1.2GB	약 160MB (−87%)
핵심 실행 코드	약 24K LOC	약 11.4K LOC (−53%)

한국 주식 SDK와 연구용 시그널 생성처럼 Python 생태계 의존이 큰 영역은 의도적으로 남겼습니다.

결과

1,600+ 계좌, 거래소·증권사 6곳, 24/7 운영, 월평균 거래대금 약 5천억 원. 주문 집행 영역에서 운영 로그로 처리 경로를 추적하지 못한 메시지는 0건이었습니다.

RabbitMQ · Celery · FastAPI · Rust · tokio · sqlx · MySQL · Redis · Prometheus

8. 레거시 데이터 파이프라인을 Airflow 플랫폼으로 전환

STOA Investment · 2026.03 운영 전환 · 설계·구현 전담

문제

수집·적재 로직이 레거시 코드베이스 2곳에 흩어져 있었고, 스케줄링 · DB 접근 · 파일 저장 · 외부 API 호출 · 검증이 한 스크립트에 묶여 있었습니다. 새 데이터 소스 추가, 백필, 스토리지 전환이 매번 코드 수정이었습니다.

영역	기존	변경	이유
오케스트레이션	Celery Beat 주기 실행	Airflow DAG · Sensor · TaskGroup	의존성·실행 이력·재시도·백필을 플랫폼에서 관리
DB 접근	배치별 연결 생성	공통 세션 팩토리 + repository	커넥션 풀·트랜잭션 정책 일원화
스토리지	환경별 파일 경로	S3 · Azure Blob · Local 어댑터	운영·재처리 환경이 같은 계약으로 동작
품질	수작업 확인	row count · schema · key/날짜 coverage · 값 범위 게이트	기준 미달 데이터는 적재하지 않고 검토 상태로 분리

Managed Airflow 대신 자체 운영을 택했습니다. 기존 Docker Compose 인프라와 배치 규모에서는 비용 대비 통제 가능성이 더 높았기 때문입니다. 전환 후 레거시 파이프라인 2종을 폐기했습니다.

Apache Airflow SQLAlchemy Docker Compose AWS S3 Azure Blob

9. 서비스 매출 40%에 기여한 구독형 운용 상품 — 기획부터 엔진까지

STOA Investment · 2024–2026.06 · 상품 기획·전략 엔진·라이브 운영 전담

신규·라이트 유저가 전략 설계의 학습 비용 때문에 실제 운용까지 가지 못한다는 이탈 원인을 짚고, 기관형 자산배분 전략을 3-클릭으로 구독하는 상품으로 기획했습니다. 이어 그 엔진이 시뮬레이션과 실제 계좌에서 같은 결과를 내도록 설계·운영했습니다. 상품은 1,600+ 계좌 운용 인프라에 적용되어 AUM 500억 원 이상, 서비스 매출 40% 기여로 이어졌습니다.

- **Point-in-time 데이터** — 거시 지표는 발표일 기준으로 사용 가능 시점을 맞추고, 시그널 다음 거래일 체결을 원칙으로 해 미래 정보 유입을 막았습니다.
- **상태 3계층** — 최신 최적화 결과 · 직전 승인 비중 · 실제 포지션을 따로 저장하고, 승인 비중과의 편차가 임계값 이하면 리밸런싱을 보류해 회전율을 관리했습니다.
- **모듈 경계** — 국면 판별 · 공분산 전처리(EWM, Ledoit-Wolf, RMT, PD 보정) · 리스크 예산 · 꼬리위험 오버레이를 교체 가능한 모듈로 나누고, 주문 side effect는 7번 실행 계층에 위임했습니다.
- 결과: 시뮬레이션 4.15년 구간과 라이브 1.36년 구간의 정합성을 검증했습니다.

Python | Backtest | Point-in-time